



Universidad Carlos III
Curso Sistemas Distribuidos
2025-26
Práctica final

Práctica final

Fecha: **03/05/2026** - ENTREGA: **10/05/2026**

GRUPO: **81**

Alumnos: **María Arias Rodríguez; NIA: 100522272 / Jorge Ignacio Castañeda Vallenilla; NIA: 100522273**

Índice

Descripción del código.....	3
Servidor de mensajería.....	3
Gestión de concurrencia e inicio.....	3
Protocolo y manejo de peticiones.....	3
Estructura de datos y persistencia.....	5
Base de datos.....	5
Cliente.....	6
Interfaz del usuario.....	6
Operaciones del protocolo.....	7
Comunicación de bajo nivel.....	8
Gestión del hilo de escucha.....	8
Servicio web.....	9
RPC.....	9
Forma de compilar.....	9
Obtención del ejecutable.....	9
Forma de ejecutar.....	9
Batería de pruebas.....	10
Conclusiones.....	12
Problemas encontrados.....	12
Soluciones.....	13
Opiniones personales.....	13

Descripción del código

Servidor de mensajería

El servidor utiliza sockets TCP junto con un modelo de concurrencia basado en hilos POSIX. Su estructura general es la de un bucle principal que acepta conexiones entrantes y delega su procesamiento a hilos secundarios. A continuación explicaremos las diferentes funciones implementadas.

Gestión de concurrencia e inicio

En este apartado, contamos con una única función:

- **main:** Es el punto de entrada del programa. Se encarga de inicializar la base de datos cargando los datos persistidos en disco, leer la variable de entorno LOG_RPC_IP necesaria para el servicio RPC, crear y configurar el socket TCP del servidor, y arrancar el bucle de aceptación de conexiones. Para controlar la concurrencia máxima, se utiliza un semáforo que limita el número de hilos activos simultáneos a MAX_HILOS. Por cada conexión aceptada, se lanza un hilo mediante pthread_create y se le pasa la información del cliente como argumento.

Protocolo y manejo de peticiones

Función para atender peticiones:

- **atender_peticion:** Es la función que ejecuta cada hilo lanzado desde main. Lee la primera cadena enviada por el cliente, que identifica la operación solicitada, y redirige el flujo al manejador correspondiente. Las operaciones reconocidas son: REGISTER, UNREGISTER, CONNECT, DISCONNECT, USERS, SEND y SENDATTACH. Al finalizar, cierra el socket y libera el semáforo para permitir que un nuevo hilo pueda atender otra conexión.

Funciones de registro:

- **my_register:** Maneja el registro de usuarios.
- **my_unregister:** Maneja la baja de usuarios.

Ambas leen el nombre de usuario del socket, invocan la función correspondiente de la base de datos (insert_user o delete_user) y devuelven un byte de respuesta al cliente indicando el resultado de la operación.

Funciones de conexión:

- **my_connect:** Gestiona la conexión de un usuario al sistema. Lee el nombre del usuario y el puerto de escucha que ha abierto el cliente para recibir mensajes. Registra estos datos en la base de datos llamando a connect_user. Si la conexión es exitosa, recorre la lista de mensajes pendientes del usuario y los entrega uno a uno mediante my_send_connected, enviando

también la confirmación al remitente de cada mensaje con `my_send_ACK`. Todo el acceso a la base de datos durante esta fase se protege con un mutex.

- **my_disconnect:** Gestiona la desconexión. Lee el nombre del usuario del socket y llama a `disconnect_user`, que elimina la IP y el puerto de escucha del registro del usuario y lo marca como desconectado. El resultado se devuelve al cliente.

Función para la lista de usuarios conectados:

- **my_users:** Atiende la solicitud de lista de usuarios conectados. Comprueba que el usuario solicitante esté conectado, obtiene la lista de usuarios activos mediante `return_connected` y la envía al cliente en el formato usuario - IP - puerto, precedida del número total de usuarios conectados.

Funciones de envío:

- **my_send:** Es el manejador unificado para las operaciones `SEND` y `SENDATTACH`, distinguiéndose mediante un parámetro de tipo entero. Lee los datos del mensaje (remitente, destinatario, texto y, en caso de `SENDATTACH`, nombre de fichero), comprueba que ambos usuarios existen en la base de datos, asigna un identificador incremental al mensaje, lo almacena y devuelve el código de respuesta junto con el identificador. Si el destinatario está conectado en ese momento, el mensaje se entrega inmediatamente llamando a `my_send_connected` y se notifica al remitente con `my_send_ACK`. De lo contrario, el mensaje queda en cola hasta que el destinatario se conecte.
- **my_send_connected:** Establece una nueva conexión TCP hacia el hilo de escucha del cliente destinatario y le envía el mensaje siguiendo el protocolo definido, primero la operación (`SEND_MESSAGE` o `SEND_MESSAGE_ATTACH`), después el remitente, el identificador, el texto y, si corresponde, el nombre del fichero adjunto.
- **my_send_ACK:** Notifica al remitente de que su mensaje ha sido entregado correctamente. Para ello abre una conexión TCP hacia el hilo de escucha del remitente y le envía la operación `SEND_MESS_ACK` (o `SEND_MESS_ATTACH_ACK` en el caso de `SENDATTACH`) junto con el identificador del mensaje y, si procede, el nombre del fichero.

Función de registro RPC:

- **rpc_log:** Es la función que el servidor utiliza como cliente RPC para notificar al servicio de registro cada operación realizada. Recibe el nombre del usuario, la operación ejecutada y, en el caso de `SENDATTACH`, el nombre del fichero adjunto (cadena vacía en cualquier otro caso). Comprueba primero que la variable de entorno `LOG_RPC_IP` esté definida, si no lo está, omite el registro sin interrumpir el flujo normal del servidor. En caso contrario, crea una conexión TCP al servidor RPC mediante `clnt_create`, invoca la función remota `log_operation_1` con los parámetros recibidos y destruye el cliente RPC al finalizar. Esta

función se llama desde cada uno de los manejadores de operación del servidor para mantener un registro centralizado de la actividad del sistema.

Estructura de datos y persistencia

Funciones de creación de sockets:

- **create_socket:** Función auxiliar que encapsula la creación de un socket TCP a partir de una IP y un puerto.
- **configure_socket:** Funciones auxiliares que encapsulan la configuración de la estructura sockaddr_in a partir de una IP y un puerto.

Ambas se utilizan tanto para el socket principal de escucha del servidor como para las conexiones salientes servidor-cliente.

Funciones de comunicación de bajo nivel:

- **sendMessage:** Garantiza el envío completo de un buffer de bytes, ya que write no garantiza escribir todos los datos en una sola llamada.
- **readMessage:** Lee byte a byte hasta encontrar el carácter nulo de fin de cadena.
- **readCode:** Lee exactamente un byte del socket, utilizado para los códigos de respuesta.
- **sendNumber:** Convierte un entero a su representación en cadena de caracteres y lo envía.

Base de datos

La base de datos está implementada en C como una lista doblemente enlazada de usuarios (user_list_t), donde cada nodo (user_t) contiene el nombre, la dirección IP y el puerto de escucha cuando está conectado, el estado de conexión, el contador de identificadores de mensaje y una lista enlazada de mensajes pendientes (message_t). El acceso concurrente se protege mediante un mutex (lock_db).

Funciones de creación/eliminación de usuarios:

- **insert_user:** Inserta nodos de la lista de usuarios, comprobando previamente si el usuario existe.
- **delete_user:** Elimina nodos de la lista de usuarios, comprobando previamente si el usuario existe. También libera todos los mensajes pendientes del usuario antes de eliminarlo.

Ambas funciones persisten el nuevo estado en disco llamando a save_db.

Funciones de conexión/desconexión de usuarios:

- **connect_user:** Actualiza el estado de conexión de un usuario almacenando la IP y el puerto de escucha y marcando al usuario como conectado.
- **disconnect_user:** Actualiza el estado de conexión de un usuario, pero hace el proceso inverso a connect_user.

Ambas mantienen actualizado el contador de usuarios conectados de la lista.

- **return_connected:** Devuelve un array de punteros a copias de los nodos de usuarios conectados en ese momento. Las copias son necesarias para evitar problemas si otro hilo modifica la lista mientras se está enviando la información al cliente.

Funciones de información de los usuarios:

- **search_user:** Recorre la lista de usuarios buscando uno por nombre y devuelve un puntero a su nodo. Es una función que no adquiere el mutex, por lo que se espera que sus llamantes lo hayan hecho antes.
- **check_user_status:** Comprueba si un usuario existe y, en caso de que exista, si está conectado o no. Devuelve 0 si está conectado, 1 si está desconectado y 2 si no existe.

Funciones de gestión de la cola de mensajes de cada usuario:

- **insert_message:** Añade un nuevo mensaje al final de la cola.
- **pop_first_message:** Extrae y elimina el primer mensaje de la cola.
- **pop_last_message:** Extrae y elimina el último mensaje de la cola.

Ambos son utilizados en distintos contextos para respetar el orden de entrega.

- **insert_first_message**
- **insert_last_message**

Ambos reinsertan mensajes en caso de fallo de entrega.

Funciones de la base de datos:

- **load_db:** Se ejecuta al iniciar el servidor y lee los ficheros db_users.txt y db_messages.txt para reconstruir el estado en memoria.
- **save_db:** Se llama tras cada modificación y vuelca el estado actual a dichos ficheros, garantizando que los datos no se pierdan si el servidor se reinicia.

Ambas se encargan de la persistencia.

Cliente

El cliente está implementado en Python como una clase estática. Mantiene el estado de la sesión (usuario conectado, lista de usuarios, dirección del servidor) en atributos de clase y expone los métodos correspondientes a cada operación del protocolo.

Interfaz del usuario

Función del bucle principal de interacción:

- **shell:** Lee comandos del usuario por consola, los analiza sintácticamente y llama al método correspondiente. Soporta los comandos REGISTER, UNREGISTER, CONNECT, DISCONNECT, USERS, SEND, SENDATTACH, GETFILE y QUIT.

Operaciones del protocolo

Funciones de registro:

- **register:** Abre una conexión TCP con el servidor, envía la cadena REGISTER seguida del nombre de usuario, lee el byte de respuesta y muestra por consola el resultado: REGISTER OK si el registro fue exitoso, USERNAME IN USE si el nombre ya estaba en uso, o REGISTER FAIL en cualquier otro caso.
- **unregister:** Abre una conexión TCP con el servidor, envía la cadena UNREGISTER seguida del nombre de usuario y lee el byte de respuesta. Si la operación tiene éxito, limpia además el usuario almacenado en _user. Muestra por consola UNREGISTER OK, USER DOES NOT EXIST o UNREGISTER FAIL según el código recibido.

Funciones de conexión:

- **connect:** Antes de enviar la solicitud de conexión al servidor, llama a _server_open para crear el socket de escucha del cliente y obtener el puerto asignado. Ese puerto se incluye en el mensaje de conexión para que el servidor sepa dónde enviar los mensajes posteriores. Si la operación tiene éxito, se registra el usuario en _user.
- **disconnect:** Abre una conexión TCP con el servidor, envía la cadena DISCONNECT seguida del nombre de usuario y lee el byte de respuesta. Independientemente del resultado, cierra también el hilo de escucha llamando a _server_close. Muestra por consola DISCONNECT OK, DISCONNECT FAIL, USER DOES NOT EXIST, DISCONNECT FAIL, USER NOT CONNECTED o DISCONNECT FAIL según corresponda.

Función de consulta de usuarios:

- **users:** Envía la solicitud de lista de usuarios conectados y parsea la respuesta, almacenando la información de cada usuario (nombre, IP y puerto) en el diccionario _user_list. Esta estructura se utiliza después por getFile para saber dónde conectarse al solicitar un fichero.

Funciones de envío de mensajes:

- **send:** Abre una conexión TCP con el servidor, envía la cadena SEND seguida del remitente, el destinatario y el mensaje previamente normalizado mediante _normalize_message. Si el servidor responde con éxito, lee además el identificador asignado al mensaje y lo muestra en el formato SEND OK - MESSAGE <id>.
- **sendAttach:** Funciona de forma idéntica a send pero incluye también el nombre del fichero adjunto en el mensaje enviado al servidor. En caso de éxito muestra SENDATTACH OK - MESSAGE <id>, y los correspondientes mensajes de error en caso contrario.

Función de transferencia de ficheros:

- **getFile:** Implementa la transferencia de ficheros entre clientes. Busca en `_user_list` la dirección del usuario que tiene el fichero, se conecta directamente a su hilo de escucha (sin pasar por el servidor), le envía la operación `GET_FILE` junto con el nombre del fichero y recibe el contenido en bloques, escribiéndolo en el fichero local indicado.

Comunicación de bajo nivel

Funciones de envío y recepción:

Funciones de bajo nivel equivalentes a las del servidor en C.

- **_sendMessage:** Asegura que las cadenas siempre se envíen terminadas en `\0` y truncadas a 255 caracteres.
- **_recieveMessage:** Lee byte a byte hasta el nulo.
- **_recieveCode:** Lee un único byte de respuesta.

Funciones de gestión de conexión:

- **_client_open:** Crea un socket TCP y establece una conexión con la dirección IP y puerto indicados como parámetros. Devuelve el socket resultante para que la función llamante pueda usarlo para enviar y recibir datos. En caso de error lanza una excepción.
- **_client_close:** Cierra el socket pasado como parámetro, liberando los recursos asociados a la conexión. Se llama al finalizar cada operación del protocolo.

Función de normalización:

- **_normalize_message:** Antes de enviar cualquier mensaje de texto, esta función llama al servicio web SOAP de normalización (a través de la librería zeep) para eliminar espacios en blanco repetidos del contenido. Si el servicio no está disponible, se muestra un aviso y el mensaje se envía tal cual.

Gestión del hilo de escucha

Funciones del ciclo de vida del hilo:

- **_server_open:** Crea un socket de escucha en un puerto libre (asignado automáticamente por el sistema operativo), lanza un hilo secundario que ejecuta `_server_listen` y devuelve el puerto asignado.
- **_server_close:** Detiene el hilo de escucha estableciendo la variable `_local_server_open` a `False`, espera a que el hilo termine limpiamente mediante `join` y cierra el socket de escucha. Se llama al desconectarse o al salir del programa con `QUIT`.

Función de escucha:

- **_server_listen:** Es el bucle del hilo: acepta conexiones entrantes del servidor y las procesa según la operación recibida, que puede ser `SEND_MESSAGE`, `SEND_MESS_ACK`,

SEND_MESSAGE_ATTACH, SEND_MESS_ATTACH_ACK o GET_FILE, mostrando el resultado al usuario o sirviendo el fichero solicitado.

Servicio web

Implementado con la librería spyne, expone un servicio SOAP en el puerto 8000. Contiene un único método, normalize, que recibe una cadena de texto, aplica una expresión regular para sustituir cualquier secuencia de espacios o tabuladores por un único espacio, y devuelve la cadena resultante. El cliente Python lo consulta antes de cada envío de mensaje.

RPC

El servicio de registro está basado en ONC-RPC. La interfaz se define en el fichero log.x, que declara el programa LOG con la versión LOGVER y una única operación: log_operation, que recibe el nombre del usuario, la operación realizada y, en el caso de SENDATTACH, el nombre del fichero. La implementación del servidor RPC (log_server.c) imprime esta información por pantalla cada vez que recibe una llamada. El servidor de mensajería actúa como cliente RPC e invoca esta operación tras procesar cada petición, utilizando la dirección del servidor RPC indicada en la variable de entorno LOG_RPC_IP.

Forma de compilar

Obtención del ejecutable

Para compilarlo, necesitamos las condiciones del entorno docker propuesto para la práctica de AG.

Para cada dispositivo o working directory ejecutamos en la terminal (una vez por sitio donde se necesiten los ejecutables), dentro de cualquiera de los contenedores lo siguiente:

```
&> make
```

Forma de ejecutar

Una vez compilado, dependiendo del rol que vaya a tomar el contenedor docker actual, tomamos uno de los 3 caminos para establecer los roles.

1) CLIENTE: Ejecutar el script que instala pip3, las dependencias de spyne, zeep y lxml, inicia el servidor web service de SOAP11 y finalmente ejecuta el cliente que apunta al servidor cuya dirección está especificada como parámetros.

```
&> ./prep_client [DIRECCIÓN DEL SERVIDOR] [PUERTO DEL SERVIDOR]
```

2) SERVIDOR: Establece la variable de entorno y ejecuta directamente el servidor

```
&> ./prep_servidor [DIRECCIÓN DEL SERVIDOR RPC] [PUERTO DE ALOJAMIENTO DEL SERVIDOR]
```

Una vez compilado, dependiendo del rol que vaya a tomar el contenedor docker actual, tomamos uno de los 3 caminos para establecer los roles.

3) SERVIDOR DE LOGS RPC: No necesita más dependencias de las que hay instaladas ya en el docker, ejecutamos directamente

```
&> ./log_client
```

Batería de pruebas

Ejecución	Descripción	Resultado esperado	Resultado final
REGISTER / UNREGISTER	Probar todos los casos de uso cruzando entre máquinas para intentar romperlo	Hay sincronía y coherencia, no debería romperse puesto que hay mutex	Tal y como esperábamos, los casos funcionan bien entre clientes
CONNECT / DISCONNECT	""	""	"" + tenemos la intuición de que si combinamos esta prueba y la anterior algo puede salir mal
REGISTER / UNREGISTER + CONNECT / DISCONNECT	Vamos a intentar romper haciendo órdenes más ilógicos y erráticos	Creemos que hay algo que se podrá romper casi con total certeza	En efecto, puedes hacer unregister de un usuario conectado sin necesidad de tener la misma IP, no es tan fácil de solucionar pues un registro no tiene por qué haberse registrado desde la misma IP
SEND / SENDATTACH	Mandar un mensaje con los dos usuarios conectados	Esperamos que los mensajes lleguen sin problemas	Funciona a la perfección, revienta en cuanto se encuentra una tilde
SEND / SENDATTACH	Mandar mensaje a un usuario desconectado y conectarlo	Al conectarse recibe los mensajes perdidos	Resultado esperado + caso particular, si dejas conectados a los usuarios en el servidor, pero cierras y abres el cliente. Lanza

			un USER ALREADY CONNECTED de la información de la sesión previa. Al intentar reenviar los mensajes el usuario se desconecta de forma automática, lo cual retorna el flujo a un estado sano
SEND / SENDATTACH	Mandar un mensaje desde un usuario desconectado	El handler de Python lo detecta al vuelo y como no sabe de parte de quien mandar el mensaje no sucede nada	El handler da feedback del problema
USERS	Uso cuando el usuario está conectado	Funcionamiento normal	Funcionamiento normal, retorna la lista de usuarios conectados con sus ips y puertos correspondientes
USERS	Uso cuando el usuario está desconectado	No pasa la petición	La petición sí pasa, pero no retorna con éxito, se recibe cliente con nombre None, lo cual puede generar fallo si hay un cliente que se llame None
USERS	Caso raro, conectar al usuario, reiniciar cliente y volver a conectar	No debería funcionar	Genera el hilo, permite consultar y almacenar los resultados de las direcciones de los usuarios, pero el thread de escucha no está actualizado en el servidor (el usuario ve su dirección antigua)
USERS + CONNECT + UNREGISTER	Conectar dos usuarios, unregister de uno de ellos mientras está conectado	Esperamos que la función de delete se percate de que esté conectado y lo reste de la lista de usuarios conectados	El servicio de delete_user de la db elimina satisfactoriamente al usuario a sus mensajes y su estado de conexión para que

			no se rompa nada
GETFILE	Mandar un fichero pequeño (1 paquete)	Esperamos que funcione sin problemas	En efecto, el fichero se transfiere en un momento
GETFILE	Mandar un fichero más grande (2+ paquetes)	""	""
GETFILE	Hacer un getfile a un usuario desconectado	No debería funcionar	No funciona
GETFILE	Explotación del caso raro de USERS	Creemos que se puede mandar un fichero dependiendo del caso y de la fidelidad de los datos de direccionamiento recibidos por USERS.	Hay todo tipo de casos súper curiosos si trasteas lo suficiente con este exploit y abusas las versiones de las estructuras USERS no actualizadas de cada cliente. Todas las combinaciones son posibles. Ej: Recibir un fichero de un usuario correctamente conectado en la DDBB, sin tener tus datos reales del client-side-server en el servidor.

Conclusiones

Problemas encontrados

La práctica, desarrollada en unas 70-100 horas ha presentado todo tipo de problemas, desde gestión inicial de punteros y cadenas de caracteres en C. Hasta errores más complejos que involucran la comunicación entre distintas máquinas. Incluso hemos tenido alguna reconfiguración de router de por medio debido a las características del entorno de prueba entre ordenadores. Mencionaré brevemente los problemas más destacables.

- 1) Envío de bytes contra envío de cadenas: Este problema estuvo gracioso, empezamos mandando todo como cadenas de caracteres sin haber terminado de entender que un byte no podía llevar el terminador de cadenas '\0' puesto que serían 2 bytes. Estandarizar todo el protocolo de envío de mensajes estaba siendo complejo.
- 2) Falta inicial de comentarios.

- 3) Diseño de la DB: No teníamos claro cómo queríamos implementar la base de datos de nuestro servidor, ni a qué aplicarle mutex dentro de las funciones o aplicarla fuera, en cada una de sus llamadas.
- 4) Código difícil de entender: Llegamos a un momento donde los protocolos no se entendían bien y era sumamente difícil identificar los fallos que buscábamos.
- 5) Falta de un plan: Hemos invertido mucho tiempo en diversos refactors de código que han sido muy útiles, pero sí predecibles por no planear ANTES DE PONERNOS A PROGRAMAR, los flujos de las funcionalidades y la arquitectura de manera conceptual.

Soluciones

Solución a los problema destacables:

- 1) Funciones mejor definidas: Tomar toda la gestión de cadenas y transformarla a algo más fácil de entender, separando funciones de envío de Números como cadenas, de bytes y de cadenas normales. De modo que todo el sistema se volvió considerablemente más legible, fácil de navegar y de modificar.
- 2) Parar un momento y poner comentarios a todo, retornos de funciones de la db (esto en concreto fue un antes y un después de no tener que estar mirando el mismo código para entenderlo por quinta vez en la misma sesión.
- 3) Nuevamente la estandarización de código fue lo más importante, decidir un modelo PLANIFICADO antes de picar nada de código, aún así fueron necesarias futuras modificaciones, que serían mucho más sencillas tras aplicar un refactor de importancia.
- 4) División en bloques de funcionalidades o más pequeños: Hacer todo el código repetido (como con lo de las cadenas) funciones, al comienzo fueron pocas repeticiones, pero fueron aumentando, lo cual forzó a que llegásemos a hacer lo más atómico posible.
- 5) Retiradas periódicas o cuando empezábamos nuevos apartados, entender primero y planificar qué hacer, funciones, reutilización de lo ya construido, necesidad de modificar /mejorar código ya existente...

Opiniones personales

Esta práctica es una de las más complejas que hemos tenido hasta ahora, engloba contenidos de muchas asignaturas que hemos cursado a lo largo de la carrera aplicadas a un solo proyecto. Lo hemos pasado bien haciendo la práctica, es muy útil para entender los conceptos que permiten que las aplicaciones de nuestro día a día funcionen correctamente. El significado de mandar un mensaje de Whatsapp no será lo mismo después de esto, eso seguro. Nos ha parecido una práctica completa y exigente, el modelo de comunicación que pide el enunciado para comunicar los dispositivos (y cada uno de sus roles: cliente y servidor) mediante diversos métodos ha sido todo un reto y uno que ha resultado de lo más satisfactorio superar.

Encontramos esta práctica muy útil, alineada con los contenidos enseñados a lo largo del cuatrimestre y como una forma excelente de aprender la asignatura.